

Data Ingestion Pipeline Creation Using Multi-Agent LLMs

Varsha Sowri Rajan

100001336

Supervisor: Prof. Dr.-Ing. Binh Vu

20.03.2026



Abstract

In recent times, the rise of LLMs has paved the way for building systems that can make decisions independently. Thus, resulting in the age of Agentic AI, where LLM Agents can orchestrate, build, and execute tasks within minutes. The goal, however, is to build autonomous systems that accomplish this with as little human intervention as possible. Traces of these autonomous systems already exist in the form of multi-agent personal assistants like OpenClaw, but implementing and maintaining such autonomous behavior for complex workflows still poses significant challenges.

While single-agent LLMs excel at task-specific workflows, multi-agent systems are needed for more complex, dynamic enterprise workflows. But such systems struggle to effectively overcome challenges like coordination between agents, fault detection, and task allocation. Although considerable progress has been made to ensure effective coordination and, in extension, task allocation, multi-agent systems continue to struggle with fault correction. They are often trapped in endless loops trying to validate and reset their track of operation, which results in higher exhaustion of their resources. Such problems often demand higher human intervention in order to establish a corrected course of action and proceed in the desired direction.

The goal of this thesis is to implement a multi-agent system that can generate data ingestion pipelines that are capable of detecting and repairing failures through automated validation and LLM-assisted debugging, thereby reducing the extent of human intervention.

1. Introduction

Many enterprises have adopted the use of LLM agents to increase employee productivity significantly. Efficient agents like GitHub Copilot help businesses to speed up a multitude of tasks, from setting up the entire workspace to committing code, writing issues, and raising pull requests. These single LLM-powered agents show remarkable success when working on task-specific problems such as data retrieval, process documentation, and report generation. However, enterprise-level workflows and architectures often require more complex and dynamic solutions that are beyond the expertise of a task-specific single fine-tuned agent.

This limitation is addressed by LLM-powered multi-agent systems (LaMAS), where several agents coordinate to achieve a single goal and complete complex tasks, each leveraging a

unique set of tools and skills. Enterprises are now determined to implement these architectures to solve high-impact use cases such as business process automation, data and insight extraction, and cross-functional teamwork.

1.1. Motivation

With the exponential growth of AI's capabilities, enterprises are aiming to transition their work environments accordingly. Businesses are experimenting with building robust systems that can carry out complex yet predictable workflows autonomously, enabling employees to focus on higher-value tasks such as strategy and system design. Enterprises that adopt autonomous AI aim to gain competitive advantages through faster decision-making and operational efficiency. Additionally, such autonomous systems are more scalable and do not require proportional human oversight with increased workloads. Thus, current businesses seek to implement LaMAS to execute repetitive workflows, namely data ingestion, process automation, and report generation. However, deploying these architectures in the aforementioned use cases would mean tackling some dynamic and complex workflows, and current LaMAS systems struggle with this.

1.2. Problem Statement

To deploy an end-to-end data pipeline, most LaMAS struggle with: a. orchestrating sub-agents to align with each other to prevent bottlenecks and follow a set execution plan. b. Persistent state tracking to ensure consistency. c. Handling of propagating errors. d. Recalibrating execution if any modules fail or deviate from the execution plan. Additionally, when multiple modules are dependent on each other for their completion, recovery from cascading errors is extremely complex. These scenarios often require frequent human intervention, especially when LaMAS architecture fails due to abstract or unprecedented errors, in order to successfully finish execution. Consequently, most of these autonomous workflows exhibit limited performance due to the lack of robust fault detection and reliable recovery mechanisms.

1.3. Contributions of this Work

This work focuses on building a multi-agent system capable of executing end-to-end data ingestion pipelines with minimal human intervention by combining autonomous fault detection and recovery mechanisms. The main contributions of this thesis are:

1. **Pipeline Orchestration:** The system decomposes the data ingestion process into modules and orchestrates multiple agents to ensure tasks like data retrieval, validation, fault detection, and pipeline repair are executed successfully.
2. **Error detection:** Implements self-healing and LLM-assisted debugging to identify and fix errors during pipeline execution.

3. **Persistent State:** Maintains a consistent execution state across modules to ensure robustness and avoid data bottlenecks.
4. **Evaluation:** Tests the system on end-to-end data ingestion pipelines to assess improvements in reliability, fault recovery, and reduction of human supervision.

1.4. Research Question

How can a multi-agent system be designed to autonomously detect and recover from errors during complex data ingestion pipelines while reducing the need for human intervention? We can answer this question by splitting it into the following corresponding questions:

1. How effectively can the proposed system detect failures in data ingestion pipelines under controlled error conditions?
 - We can measure this by conducting an evaluation of the system's ability to detect failures by introducing controlled error scenarios such as schema mismatches, missing data, and corrupted inputs, and measuring the accuracy of detection across these cases.
2. How effectively can the system autonomously repair detected failures using LLM-assisted debugging?
 - We can measure this by evaluating the system's ability to repair failures by measuring the success rate of LLM-assisted debugging and the number of iterations required to resolve different types of errors.
3. How does a multi-agent architecture compare to a baseline approach in terms of reliability and fault recovery?
 - We can measure this by evaluating the effectiveness of the multi-agent architecture by comparing pipeline success rates and the frequency of human intervention against a simpler baseline approach.
4. How efficient and stable is the system in handling repeated failures without entering unproductive execution loops?
 - We can measure this by evaluating the system's efficiency and stability by analyzing retry counts, occurrence of failure loops, and overall execution time across repeated failure scenarios.

2. State of the Art

Recent research on autonomous workflow execution has increasingly focused on multi-agent systems powered by Large Language Models (LLMs). Early works largely relied on single-agent systems, which performed well for task-specific operations but struggled with interdependent workflows. More recent studies have explored multi-agent coordination, where each agent handles a specific sub-task, allowing systems to tackle more complex and dynamic workflows.

In multi-agent LLM architectures, research has primarily focused on how complex workflows are broken down into smaller modules and coordinated among specialized agents, as well as how these systems detect errors, manage cascading failures, and maintain consistent execution. The two aspects below represent the core challenges in implementing reliable multi-agent systems for end-to-end data ingestion pipelines: a. Task decomposition and agent orchestration. b. Fault detection and recovery.

The following sections review traditional approaches in single-agent systems, traditional multi-agent systems, and LLM-powered multi-agent systems, highlighting their strengths, limitations, and the remaining challenges in achieving fully autonomous data ingestion pipelines.

2.1. LLM-Powered Single-Agent Systems

LLM-powered single agents extend the capabilities of a standalone LLM by autonomously perceiving and learning from the environment to evolve. The architecture of these agents is generally split into profile, memory, planning, and action [1]. The above architecture sufficiently equips them to handle end-to-end task-specific problems, like coding assistants and report generation, etc. However, to handle multi-step complex workflows, the agent requires more memory recalls, a wider context window, and better planning, which accounts for failures and recalibration.

These limitations make single-agent systems unsuitable for complex, interdependent workflows such as data ingestion pipelines, motivating the shift toward multi-agent architectures.

2.2. Traditional Multi-Agent Systems

While single-agent systems can take actions, the real benefit of agents can be harnessed when they work collaboratively with other agents [2]. Multi-agent systems can share their knowledge with other agents, can each independently execute decision-making processes, and use their shared history and information to predict possible future actions [2]. This efficiency stems from the division of labour whereby a complex task is broken down and assigned to a unique agent [5].

However, traditional multi-agent systems typically rely on deterministic rules, static workflows, and domain-specific logic, which limit their ability to adapt to dynamic and unstructured environments. These limitations highlight the need for more adaptive and intelligent agent architectures, motivating the emergence of LLM-powered multi-agent systems.

2.3. LLM-Powered Multi-Agent Systems

Models like GPT-4 show great ability to solve complex tasks across diverse fields, approaching human-level performance [6]. Early use of LLMs consisted primarily of single-purpose tasks such as text generation or analysis. However, recent advances have equipped them to interact with graphical user interfaces (GUIs) and perform complex operations, such as web browsing, app navigation, and system control [7]. Although such systems may require greater computational resources compared to single-agent approaches, they offer crucial advantages that justify the trade-off: inherent fault tolerance through agent redundancy, natural task decomposition without explicit workflow design, and organic specialisation in complex problem-solving [8].

In practice, such systems often struggle with reliable fault detection and recovery, particularly when operating in dynamic environments or executing multi-step workflows. Errors in intermediate steps can propagate across agents, leading to inefficient retry loops or incomplete task execution. This highlights the need for mechanisms that enable autonomous fault handling and system recovery, motivating the exploration of self-healing systems and LLM-assisted debugging.

2.4. Self-Healing Systems and Autonomous Debugging

Self-healing systems are designed to detect, diagnose, and recover from failures with minimal or no human intervention. These systems aim to improve reliability by continuously monitoring execution and applying corrective actions when faults are identified. According to existing literature, self-healing software systems typically operate across several stages, including fault detection, root cause analysis, and automated recovery [9]. This structured approach allows systems to maintain operational continuity even in the presence of unexpected errors.

The problem space of self-healing systems is broad and involves challenges such as identifying failure conditions, selecting appropriate recovery strategies, and ensuring that corrective actions do not introduce further inconsistencies [10]. Traditional approaches often rely on predefined rules or heuristics for failure handling, which limits their adaptability in dynamic and unstructured environments. As workflows become more complex, especially in distributed and modular systems, these rule-based mechanisms become increasingly difficult to scale and maintain.

Autonomous debugging extends the concept of self-healing by focusing specifically on identifying and correcting errors in system execution or generated code. It involves analyzing failures, generating potential fixes, and validating the corrected output without human intervention. However, traditional debugging approaches are often static and require explicit test cases or human-defined correction strategies, which restrict their effectiveness in evolving systems.

In the context of multi-agent systems, these challenges are further amplified due to the interdependencies between agents. Failures in one component can propagate across the system, making it difficult to isolate and resolve issues efficiently. This highlights the need for more adaptive approaches to self-healing and debugging that can operate in dynamic, multi-step workflows

2.5. LLM-Powered Debugging

Recent advances in large language models (LLMs) have introduced new possibilities for autonomous debugging by enabling systems to reason about errors and generate corrective actions dynamically. LLMs can analyze execution traces, identify inconsistencies, and propose fixes in natural language or code form, making them well-suited for debugging tasks in complex systems.

One key approach in this area is self-debugging through iterative refinement, where the model generates code, evaluates its correctness, and produces fixes based on observed failures. Techniques such as self-generated test cases have been shown to improve the reliability of generated code by enabling models to validate their own outputs [11]. This iterative process allows LLMs to move beyond one-shot generation toward more robust and adaptive problem-solving.

Further research has demonstrated that LLMs can be trained to improve their debugging capabilities by learning from execution feedback and explanations. For instance, methods that incorporate explanation generation alongside debugging tasks enable models to better understand the underlying causes of errors, leading to more accurate corrections [12]. Similarly, training strategies such as LEDEX focus on enhancing the model's ability to both debug and explain code, improving overall performance in failure recovery scenarios [13].

LLM-guided debugging approaches also show promise in handling complex, multi-step workflows, where errors may arise at different stages of execution. By leveraging their reasoning capabilities, LLMs can assist in identifying the root cause of failures and generating context-aware fixes, making them particularly suitable for integration into multi-agent systems.

Despite these advancements, LLM-based debugging is not without limitations. Challenges such as inconsistent reasoning, sensitivity to prompt design, and the risk of generating incorrect fixes remain open problems. However, their ability to perform adaptive, context-aware debugging makes them a strong candidate for enabling self-healing capabilities in modern software systems.

3. Expected Outcome of Thesis

The thesis is expected to produce an LLM-powered multi-agent system for data ingestion pipelines that autonomously orchestrates tasks, detects and recovers from errors, maintains a

consistent state, and reduces human intervention. It will demonstrate improved reliability, robustness, and efficiency in enterprise-scale pipeline execution.

4. Proposed Method

This thesis proposes a multi-agent system for the autonomous execution of data ingestion pipelines, aimed at minimizing human intervention while maintaining reliability and robustness. The architecture comprises the following specialized agents:

Orchestrator

- Oversees the planning and execution of the data pipeline, coordinating all sub-agents to ensure alignment with the drafted execution plan.
- Implements retry mechanisms and maintains a minimal persistent execution state to recover from task failures.

Data Retrieval Agent

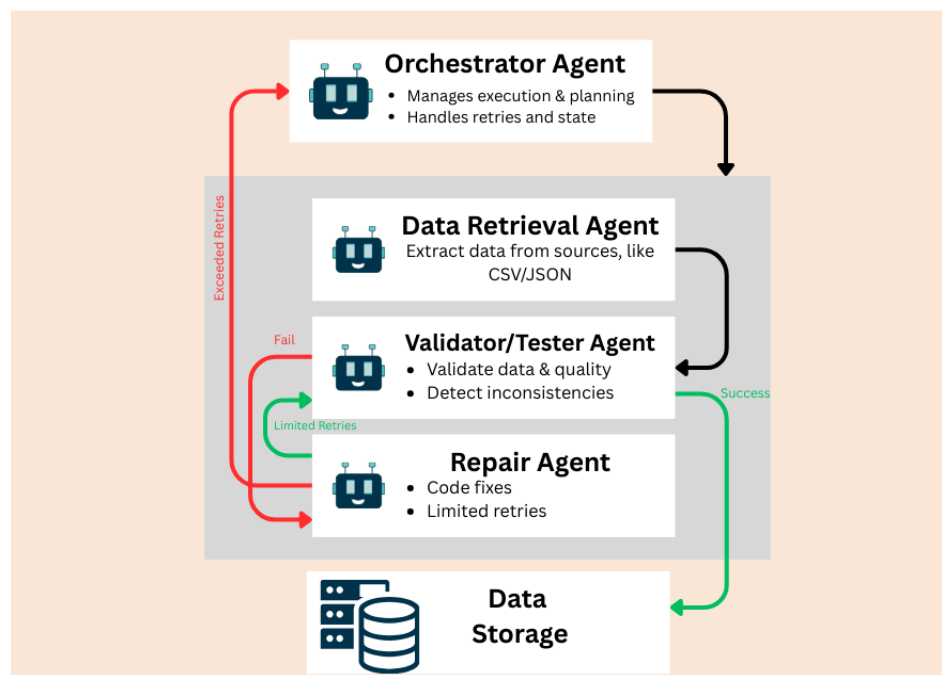
- Extracts data from selected sources, such as APIs, relational databases, or structured files (CSV/JSON).

Validator/Tester Agent

- Performs checks on retrieved data to ensure the data quality.
- Identifies inconsistencies and triggers limited reparative measures.

Repair Agent

- Executes failure recovery using LLM-assisted suggestions.
- Handles cascading errors and recalibrates execution within confined retry limits to restore pipeline execution.



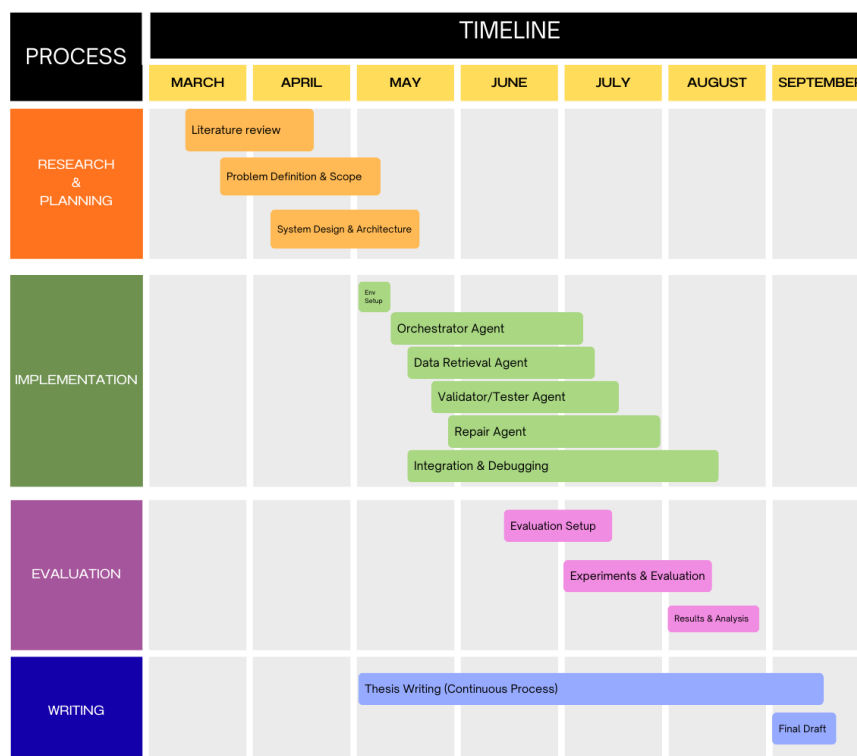
5. Evaluation

The evaluation of the proposed system will be conducted using a controlled experimental setup, as no standardized benchmarks currently exist for assessing multi-agent LLM systems in the context of data ingestion pipeline repair. A set of representative data ingestion pipelines will be defined using structured datasets (e.g., CSV and JSON formats), and controlled failure scenarios will be systematically introduced. These failures will include schema mismatches, missing or malformed data, and data type inconsistencies. The system will be evaluated based on its ability to detect and repair such failures autonomously.

To quantify performance, metrics such as pipeline success rate, failure detection rate, repair success rate, number of repair iterations, and frequency of human intervention will be measured. Each pipeline will be executed under both normal and failure-induced conditions to assess robustness. This controlled approach ensures reproducibility while enabling a focused analysis of the system’s fault detection and recovery capabilities.

6. Timeline

The following timeline of the thesis work is based primarily on the corresponding chapters of the thesis. The iterative process of communicating with the supervisor, presenting the current state of the thesis, will be set ahead.



7. Bibliography

- [1] Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y. and Zhao, W.X., 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), p.186345.
- [2] Dorri, A., Kanhere, S.S. and Jurdak, R., 2018. Multi-agent systems: A survey. *Ieee Access*, 6, pp.28573-28593.
- [3] Garcia, A.P., Oliver, J. and Gosch, D., 2010, October. An intelligent agent-based distributed architecture for smart-grid integrated network management. In *IEEE local computer network conference* (pp. 1013-1018). IEEE.
- [4] McArthur, S.D., Davidson, E.M., Catterson, V.M., Dimeas, A.L., Hatziargyriou, N.D., Ponci, F. and Funabashi, T., 2007. Multi-agent systems for power engineering applications—Part I: Concepts, approaches, and technical challenges. *IEEE Transactions on Power systems*, 22(4), pp.1743-1752.
- [5] Rezaee, H. and Abdollahi, F., 2015. Average consensus over high-order multiagent systems. *IEEE Transactions on Automatic Control*, 60(11), pp.3047-3052.
- [6] Bubeck, S., Coester, C., Eldan, R., Gowers, T., Lee, Y.T., Lupsasca, A., Sawhney, M., Scherrer, R., Sellke, M., Spears, B.K. and Unutmaz, D., 2025. Early science acceleration experiments with GPT-5. *arXiv preprint arXiv:2511.16072*.
- [7] Zhang, C., He, S., Qian, J., Li, B., Li, L., Qin, S., Kang, Y., Ma, M., Liu, G., Lin, Q. and Rajmohan, S., 2024. Large language model-brained gui agents: A survey. *arXiv preprint arXiv:2411.18279*.
- [8] Yang, Y., Peng, Q., Wang, J., Wen, Y. and Zhang, W., 2024. Llm-based multi-agent systems: Techniques and business perspectives. *arXiv preprint arXiv:2411.14033*.
- [9] Yazdanparast, Z., 2024. A Survey on Self-healing Software System. *arXiv preprint arXiv:2403.00455*.
- [10] Koopman, P., 2003, May. Elements of the self-healing system problem space. In *Workshop on Architecting Dependable Systems/WADS03*.
- [11] Chen, X., Tao, Z., Zhang, K., Zhou, C., Zhang, X., Gu, W., He, Y., Zhang, M., Cai, X., Zhao, H. and Jin, Z., 2025, July. Revisit self-debugging with self-generated tests for code generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 18003-18023).
- [12] Adnan, M., Xu, Z. and Kuhn, C.C., 2025. Large language model guided self-debugging code generation. *arXiv preprint arXiv:2502.02928*.
- [13] Jiang, N., Li, X., Wang, S., Zhou, Q., Hossain, S.B., Ray, B., Kumar, V., Ma, X. and Deoras, A., 2024. Ledex: Training LLMs to better self-debug and explain code. *Advances in Neural Information Processing Systems*, 37, pp.35517-35543.